

Observability Guide — AI Readiness Training Program

Section 1: Overview and Philosophy

Why Observability Matters in AI Systems

Traditional software has deterministic behavior — given the same input, you get the same output. AI systems are different:

- LLM responses vary based on temperature and context
- RAG retrieval quality depends on chunk quality and embedding similarity
- Agent reasoning chains are non-deterministic
- Multi-agent systems have complex state transitions

Without observability, debugging AI systems is nearly impossible. This guide establishes standards for **what to observe**, **how to log it**, and **how to trace it** across all 5 phases.

The Three Pillars

Pillar	Tool	Purpose
Logs	structlog (Python) / Serilog (.Net) / SLF4J+Logback (Java)	Record discrete events with structured metadata
Traces	LangSmith (AI calls) + OpenTelemetry (infrastructure)	Track full request flows across services and LLM calls
Metrics	OpenTelemetry (basic counters/timers)	Measure system performance over time

For this program, focus on **Logs** and **Traces** — they provide the most value for debugging AI pipelines.

Section 2: Structured Logging Standard

2.1 Mandatory Log Fields (All Phases)

Every log event must include these fields in JSON format:

```
{
  "timestamp": "2026-06-17T14:30:00.123Z",
  "level": "INFO",
  "poc_id": "POC-01",
  "phase": 1,
  "associate_id": "john.doe",
  "operation": "create_application",
  "duration_ms": 45,
  "status": "success",
  "error": null,
  "request_id": "req_abc123",
}
```

```
"extra": {}  
}
```

Field	Type	Description
timestamp	ISO8601 string	When the event occurred
level	INFO/WARN/ERROR/DEBUG	Severity
poc_id	string	e.g., "POC-01"
phase	integer	1–5
associate_id	string	Identifies whose code is running
operation	string	What operation is being performed
duration_ms	integer	How long the operation took
status	success/failure	Outcome
error	string or null	Error message if failure
request_id	string	Unique ID for tracing a request

2.2 Python — structlog Setup

```
# logging_config.py  
import structlog  
import logging  
import os  
  
def configure_logging():  
    structlog.configure(  
        processors=[  
            structlog.stdlib.filter_by_level,  
            structlog.stdlib.add_logger_name,  
            structlog.stdlib.add_log_level,  
            structlog.processors.TimeStamper(fmt="iso"),  
            structlog.processors.StackInfoRenderer(),  
            structlog.processors.format_exc_info,  
            structlog.processors.JSONRenderer(),  
        ],  
        context_class=dict,  
        logger_factory=structlog.stdlib.LoggerFactory(),  
        cache_logger_on_first_use=True,  
    )  
  
    logging.basicConfig(  
        format="%(message)s",  
        stream=sys.stdout,  
        level=logging.INFO,  
    )
```

```
# Usage
logger = structlog.get_logger()

# Bind context once per request
log = logger.bind(
    poc_id=os.getenv("POC_ID", "POC-01"),
    phase=int(os.getenv("PHASE", "1")),
    associate_id=os.getenv("ASSOCIATE_ID", "unknown"),
    request_id="req_abc123"
)

# Log an operation
import time
start = time.time()
try:
    result = do_something()
    log.info("operation_complete",
            operation="create_application",
            duration_ms=int((time.time() - start) * 1000),
            status="success",
            application_id=result.id)
except Exception as e:
    log.error("operation_failed",
            operation="create_application",
            duration_ms=int((time.time() - start) * 1000),
            status="failure",
            error=str(e))

raise
```

2.3 .Net C# — Serilog JSON Setup

```
// Program.cs
using Serilog;
using Serilog.Formatting.Compact;

Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console(new CompactJsonFormatter())
    .Enrich.WithProperty("PocId", "POC-01")
    .Enrich.WithProperty("Phase", 1)
    .Enrich.WithProperty("AssociateId",
        Environment.GetEnvironmentVariable("ASSOCIATE_ID") ?? "unknown")
    .Enrich.WithEnvironmentName()
    .CreateLogger();

builder.Host.UseSerilog();

// Usage in a controller or service:
private readonly ILogger<ApplicationService> _logger;

public ApplicationService(ILogger<ApplicationService> logger)
```

```
{
    _logger = logger;
}

public async Task<Application> CreateApplicationAsync(CreateApplicationDto dto)
{
    var sw = Stopwatch.StartNew();
    try
    {
        var app = await _repository.CreateAsync(dto);
        _logger.LogInformation(
            "Operation: {Operation} Status: {Status} Duration: {DurationMs}ms",
            ApplicationId: {AppId}",
            "create_application", "success", sw.ElapsedMilliseconds, app.Id);
        return app;
    }
    catch (Exception ex)
    {
        _logger.LogError(ex,
            "Operation: {Operation} Status: {Status} Duration: {DurationMs}ms",
            "create_application", "failure", sw.ElapsedMilliseconds);
        throw;
    }
}
```

2.4 Java — Logback JSON Setup

logback-spring.xml:

```
<configuration>
  <appender name="JSON_CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <customFields>{"poc_id":"POC-01","phase":1}</customFields>
    </encoder>
  </appender>
  <root level="INFO">
    <appender-ref ref="JSON_CONSOLE" />
  </root>
</configuration>
```

Usage in Java:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import net.logstash.logback.argument.StructuredArguments;

private static final Logger log =
    LoggerFactory.getLogger(ApplicationService.class);
```

```
public Application createApplication(CreateApplicationRequest request) {
    long start = System.currentTimeMillis();
    try {
        Application app = repository.save(mapper.toEntity(request));
        log.info("Operation complete",
            StructuredArguments.kv("operation", "create_application"),
            StructuredArguments.kv("status", "success"),
            StructuredArguments.kv("duration_ms", System.currentTimeMillis() -
start),
            StructuredArguments.kv("application_id", app.getId()));
        return app;
    } catch (Exception e) {
        log.error("Operation failed",
            StructuredArguments.kv("operation", "create_application"),
            StructuredArguments.kv("status", "failure"),
            StructuredArguments.kv("duration_ms", System.currentTimeMillis() -
start),
            StructuredArguments.kv("error", e.getMessage()));
        throw e;
    }
}
```

Section 3: LangSmith Tracing

3.1 Setup and Configuration

```
# In your .env file:
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=lsv2_pt_your_key
LANGCHAIN_PROJECT=AI-Readiness-POC-01-P2

# In your Python code – call before any LangChain imports:
from dotenv import load_dotenv
load_dotenv()
```

3.2 What Gets Auto-Traced

When `LANGCHAIN_TRACING_V2=true`, LangChain automatically traces:

- All LLM calls (input prompt, output, tokens used, latency)
- All retriever calls (query, retrieved documents, scores)
- All chain executions (each step in a chain)
- All agent actions (tool calls, reasoning steps)
- All tool invocations (tool name, input, output)

3.3 Adding Custom Metadata

```

from langsmith import traceable

# Decorate any function to add it to the trace
@traceable(
    name="rag_query",
    tags=["phase-2", "rag"],
    metadata={
        "poc_id": "POC-01",
        "phase": 2,
        "associate_id": "john.doe"
    }
)
def answer_question(question: str) -> str:
    # Your RAG logic here
    docs = retriever.invoke(question)
    response = chain.invoke({"question": question, "context": docs})
    return response

# With run metadata
from langchain_core.runnables import RunnableConfig

config = RunnableConfig(
    metadata={
        "poc_id": "POC-01",
        "phase": 3,
        "query_type": "tool_assisted"
    },
    tags=["phase-3", "agent"]
)
result = agent.invoke({"input": question}, config=config)

```

3.4 Project Naming Convention

Use this naming convention for LangSmith projects:

AI-Readiness-{POC_ID}-P{Phase_Number}

Examples:

- AI-Readiness-POC-01-P2 — POC 01, Phase 2
- AI-Readiness-POC-05-P4 — POC 05, Phase 4

Set this in your `.env`:

LANGCHAIN_PROJECT=AI-Readiness-POC-01-P2

Change this value when you move to the next phase.

3.5 Viewing Traces in LangSmith

1. Go to smith.langchain.com
2. Select your project from the left sidebar
3. Click any trace to see the full execution tree
4. Expand nodes to see: input/output, latency, token counts
5. For RAG traces: see retrieved documents and similarity scores
6. For agent traces: see each tool call and reasoning step

3.6 Debugging RAG with LangSmith

Common issues and how to diagnose:

Issue	How to Spot in LangSmith	Fix
Wrong chunks retrieved	Retriever node shows low relevance scores	Adjust chunk size or re-ingest
LLM ignoring context	Generation node shows prompt but answer doesn't use context	Improve RAG prompt template
High latency	Node shows >3s duration	Check embedding model, reduce chunk count
Empty context	Retriever node returns 0 documents	Check ChromaDB collection has data

Section 4: OpenTelemetry Spans

4.1 Required Spans Per Phase

Phase 1 — Full Stack CRUD

```
# Automatic via FastAPI instrumentation + manual for DB:

# Auto (FastAPI middleware): http.request
# Manual:

with tracer.start_as_current_span("db.query") as span:
    span.set_attribute("db.operation", "INSERT")
    span.set_attribute("db.table", "loan_applications")
    result = db.add(application)

with tracer.start_as_current_span("auth.validate") as span:
    span.set_attribute("auth.user_id", user_id)
    span.set_attribute("auth.token_valid", True)
    validate_token(token)
```

Phase 2 — RAG Application

```
with tracer.start_as_current_span("rag.document_load") as span:
    span.set_attribute("rag.source", "user_manual.md")
    span.set_attribute("rag.file_size_bytes", file_size)
    documents = loader.load()
    span.set_attribute("rag.document_count", len(documents))

with tracer.start_as_current_span("rag.chunk") as span:
    span.set_attribute("rag.chunk_size", 512)
    span.set_attribute("rag.chunk_overlap", 50)
    chunks = splitter.split_documents(documents)
    span.set_attribute("rag.chunk_count", len(chunks))

with tracer.start_as_current_span("rag.embed") as span:
    span.set_attribute("rag.embedding_model", "text-embedding-004")
    vectorstore.add_documents(chunks)
    span.set_attribute("rag.vectors_stored", len(chunks))

with tracer.start_as_current_span("rag.retrieve") as span:
    span.set_attribute("rag.query", question)
    span.set_attribute("rag.top_k", 4)
    docs = retriever.invoke(question)
    span.set_attribute("rag.retrieved_count", len(docs))

with tracer.start_as_current_span("rag.generate") as span:
    span.set_attribute("rag.model", "gemini-2.0-flash")
    span.set_attribute("rag.context_chunks", len(docs))
    answer = chain.invoke({"question": question, "context": docs})
    span.set_attribute("rag.answer_length", len(answer))
```

Phase 3 — Context Engineering

```
with tracer.start_as_current_span("agent.tool_call") as span:
    span.set_attribute("tool.name", tool_name)
    span.set_attribute("tool.input", str(tool_input))
    result = tool.invoke(tool_input)
    span.set_attribute("tool.output_length", len(str(result)))

with tracer.start_as_current_span("agent.reasoning") as span:
    span.set_attribute("agent.step", step_number)
    span.set_attribute("agent.thought", thought_text[:200])

with tracer.start_as_current_span("api.call") as span:
    span.set_attribute("api.endpoint", endpoint)
    span.set_attribute("api.method", "GET")
    response = requests.get(endpoint)
    span.set_attribute("api.status_code", response.status_code)

with tracer.start_as_current_span("agent.summarize") as span:
    span.set_attribute("summarize.input_length", len(text))
```



```
summary = summarize_chain.invoke({"text": text})
span.set_attribute("summarize.output_length", len(summary))
```

Phase 4 — MCP + Chat

```
with tracer.start_as_current_span("mcp.tool_invoke") as span:
    span.set_attribute("mcp.tool_name", tool_name)
    span.set_attribute("mcp.session_id", session_id)
    span.set_attribute("mcp.input", str(tool_input))
    result = await mcp_tool.invoke(tool_input)
    span.set_attribute("mcp.success", True)

with tracer.start_as_current_span("chat.message") as span:
    span.set_attribute("chat.session_id", session_id)
    span.set_attribute("chat.message_index", message_index)
    span.set_attribute("chat.message_length", len(message))

with tracer.start_as_current_span("mcp.response") as span:
    span.set_attribute("mcp.response_length", len(str(response)))
    span.set_attribute("mcp.tool_name", tool_name)
```

Phase 5 — Multi-Agent LangGraph

```
# In each agent node function:
with tracer.start_as_current_span(f"agent.{agent_name}.activate") as span:
    span.set_attribute("agent.name", agent_name)
    span.set_attribute("agent.input_state_keys", str(list(state.keys())))
    result = agent_logic(state)
    span.set_attribute("agent.output_keys_populated", str(list(result.keys())))

with tracer.start_as_current_span("supervisor.route") as span:
    span.set_attribute("supervisor.from_agent", current_agent)
    span.set_attribute("supervisor.to_agent", next_agent)
    span.set_attribute("supervisor.routing_reason", reason)

with tracer.start_as_current_span("graph.execute") as span:
    span.set_attribute("graph.input_node", "START")
    span.set_attribute("graph.application_id", application_id)
    final_state = graph.invoke(initial_state)
    span.set_attribute("graph.final_decision", final_state.get("final_decision"))
    span.set_attribute("graph.agents_executed", str(agents_executed))
```

Section 5: Phase-Specific Observability Checklist

Phase 1 Checklist (Required for Full Observability Score)

- ☐ `configure_logging()` called on application startup
- ☐ Every API endpoint logs: entry (with request params), exit (with response status + duration_ms)
- ☐ Authentication events logged: login success, login failure, token validation
- ☐ Database operations logged: operation type, table, duration_ms
- ☐ All unhandled exceptions logged with full stack trace
- ☐ Request ID generated per request and included in all logs
- ☐ Structured log fields: `poc_id`, `phase`, `associate_id` present in every log line
- ☐ FastAPI OTEL auto-instrumentation enabled: `FastAPIInstrumentor.instrument_app(app)`
- ☐ `db.query` spans created for all database operations
- ☐ `auth.validate` spans created for token validation

Phase 2 Checklist

- ☐ `LANGCHAIN_TRACING_V2=true` set in environment
- ☐ `LANGCHAIN_PROJECT` set to `AI-Readiness-{POC_ID}-P2`
- ☐ LangSmith trace created for every RAG query (visible in LangSmith UI)
- ☐ OTEL spans: `rag.document_load`, `rag.chunk`, `rag.embed` during ingestion
- ☐ OTEL spans: `rag.retrieve`, `rag.generate` during query
- ☐ Each span has required attributes (see Section 4)
- ☐ Ingestion log: number of chunks created, embedding time
- ☐ Query log: question, number of retrieved chunks, generation time
- ☐ LangSmith trace contains retrieved documents

Phase 3 Checklist

- ☐ `LANGCHAIN_PROJECT` updated to P3
- ☐ Every tool call logged: tool name, input, output, duration_ms
- ☐ `agent.tool_call` OTEL span created per tool invocation
- ☐ `api.call` span created for every Phase 1 API call from a tool
- ☐ Summarization events logged: input_length, output_length, duration
- ☐ Agent reasoning steps logged (thought content truncated to 200 chars)
- ☐ LangSmith trace shows full ReAct reasoning chain

Phase 4 Checklist

- ☐ `LANGCHAIN_PROJECT` updated to P4
- ☐ MCP server startup logged: port, tools registered
- ☐ Every MCP tool invocation logged: tool_name, session_id, input, output, duration_ms
- ☐ Chat session ID generated on session start, included in all logs
- ☐ `mcp.tool_invoke` OTEL span per MCP tool call
- ☐ `chat.message` OTEL span per user message
- ☐ LangSmith trace includes chat session ID in metadata

Phase 5 Checklist

- ☐ `LANGCHAIN_PROJECT` updated to P5
- ☐ Graph execution logged: start, which agents ran, final decision
- ☐ Each agent activation logged: agent name, input state summary, output state keys

- ☐ Supervisor routing decisions logged: from_agent, to_agent, reason
 - ☐ `graph.execute` OTel span wrapping entire graph run
 - ☐ `agent.{name}.activate` OTel span per agent node execution
 - ☐ `supervisor.route` OTel span per routing decision
 - ☐ LangSmith shows multi-agent trace tree with all agent sub-traces
-

Section 6: Debugging with Observability

6.1 Diagnosing RAG Issues

Problem: RAG returns wrong answers

1. Open LangSmith → find the failing trace
2. Click the **Retriever** node — check retrieved documents
3. If wrong docs retrieved: chunk size may be too large, or query needs rewriting
4. Click the **LLM** node — check the full prompt sent to Gemini
5. If context looks right but answer is wrong: improve the RAG prompt template

Problem: RAG returns "I don't know" for everything

1. Check logs for `rag.retrieve` span — is `retrieved_count` = 0?
2. If yes: ChromaDB collection might be empty — re-run ingestion
3. If `retrieved_count` > 0: check similarity scores — are they very low (<0.3)?
4. Low scores mean your question vocabulary doesn't match the document vocabulary — try re-phrasing

6.2 Diagnosing Agent Tool Issues

Problem: Agent calls wrong tool

1. Open LangSmith → find the trace → look at the **Agent** node
2. Check the tool selection reasoning in the agent's thought
3. If tool descriptions are ambiguous, make them more specific
4. Add examples to tool descriptions: "Use this when the user asks about X, Y, or Z"

Problem: Tool returns error

1. Check logs for `api.call` span — what `status_code` was returned?
2. If 404: the entity ID doesn't exist in your Phase 1 database
3. If 500: the Phase 1 API has a bug — check Phase 1 logs
4. If connection error: Phase 1 API is not running — start it first

6.3 Reading Structured Logs in VS Code

Install the **JSON Log Viewer** extension or use the built-in output panel. To pretty-print your logs:

```
# Pipe logs through jq for readable output
uvicorn app.main:app --log-level info 2>&1 | jq '.'

# Filter logs for a specific operation
```

```
uvicorn app.main:app 2>&1 | jq 'select(.operation == "create_application")'

# Filter for errors only
uvicorn app.main:app 2>&1 | jq 'select(.level == "error")'
```

6.4 Common Observability Mistakes

Mistake	Problem	Fix
Not calling <code>load_dotenv()</code>	LangSmith env vars not loaded	Call <code>load_dotenv()</code> at the very start of your main module
Hardcoding <code>poc_id</code> vs reading from env	Works locally, breaks during evaluation	Always read from <code>os.getenv("POC_ID")</code>
Missing <code>duration_ms</code>	Evaluator can't verify performance	Always wrap operations in <code>time.time()</code> before and after
Logging inside a tight loop	Thousands of log lines per second	Log aggregates, not individual iterations
Not setting <code>LANGCHAIN_PROJECT</code>	All traces go to default project	Set the env var before running any Phase 2+ code